Memory over time: an object and its timeline

Our bitemporal memory vs Graphiti (open-source core of Zep) vs Mem0 OSS · live measurements, 5 Nov 2026 · Anton Dziatkovskii

The problem we test

An object changes: a hotel room is empty, then a guest checks in, then checks out; a sales lead is hot, then cools down. A good memory must (1) keep the whole history instead of overwriting it, (2) close the old fact with a date when a new one arrives, (3) answer "what was true on date D", and (4) undo a fact that was recorded by mistake, bringing the previous one back.

Result 1: room 402 scenario (deterministic, no LLM in the loop)

Scenario	Our `fact_history`	Graphiti 0.31.2 core	Mem0 OSS 2.2.1
S1. Four guests in a row: who was in the room on date D?	pass	pass	old and new value stored side by side (our 3 Nov canary)
S2. A late fact arrives for the middle of the timeline	pass	pass	not tested yet
S3. A record turns out to be wrong and is retracted	pass: previous guest comes back	fail: closing date is written into the old fact, no retract operation, the room shows empty	not tested yet
Score	3 / 3	2 / 3	**1 canary, full run pending**

Why we pass S3: in our design the end of a fact is not stored, it is derived at read time from the next fact in the chain, so retracting a mistake automatically revives the previous value. Graphiti, like ours, uses two time axes (valid time and record time).

Result 2: real CRM data (15 real changes of lead status overnight, 30 questions "what was it on date D")

System	Input	Answers correct	Latency
Ours: SQLite triggers write every change into an append-only history table, inside the same transaction	structured rows	30 / 30	0.01 ms per question, 0 LLM calls
Graphiti 0.31.2 with GLM-5.3 as the extraction LLM, Kuzu graph	the same 15 changes as plain-text episodes	12 / 30 via search	~24 s per episode to ingest (LLM extraction)

Fair reading: Graphiti did extract all 30 episodes and correctly closed old facts with dates (example: Lead A "band Now" got an end date the moment "This month" arrived). Most misses came from retrieval: in this smoke run we used a simple word-hash embedder instead of a real embedding model, so search often returned another lead's facts. A run with a real embedder (multilingual-e5) and a direct timeline check without search is in progress.

What this means

- For data that is already structured (CRM, job pipeline), a trigger-based history in the existing database is exact, instant and free. An LLM memory layer adds cost and recall errors there.

- Graphiti's strength is the step we do not have yet: turning raw text (calls, chats, notes) into dated facts automatically. That is what we want to borrow.
- Retracting a mistaken fact is a real gap in Graphiti's core and a natural question for any memory vendor.

Questions for Mem0

1. When a user's preference changes ("coffee" then "tea since Nov 2"), does Mem0 keep both with dates, or overwrite? Can I ask "what did they prefer in October"?
2. How do you undo a memory that was extracted by mistake, and does the previous value come back?
3. What do you recommend for data that is already structured: ingest it as text, or keep it in our DB and let Mem0 only handle conversations?
4. Which benchmark do you trust for temporal questions (knowledge updates, "as of" questions), and what is your number on it?

Sources: our test scripts and receipts in the repo (`fact_history.py` , `timeline_H35889.py` , `our_side_asof.py` , `graphiti_glm_asof.py`). Lead names anonymised.